

Python Style Guide

1. Code Layout

- **Indentation:** Use **4 spaces per indentation level** (never tabs).
 - **Line length:** Limit lines to **79 characters**.
 - **Blank lines:**
 - One blank line to separate code blocks
 - **Imports:**
 - One import per line.
-

2. Naming Conventions

Type	Convention	Example
Functions, variables	lowercase_with_underscores	calculate_total()
Constants	ALL_CAPS_WITH_UNDERSCORES	MAX_SCORE = 100

3. Comments & Docstrings

- **Comments** should be full sentences and explain *why*, not *what*.
- **Inline comments:** Separate by *two spaces* from code.

```
x = x + 1 # Increment counter
```

- **Docstrings:** Use triple quotes (`""" """`) for all public modules, functions, classes, and methods.

```
def greet(name):  
    """Return a greeting message for the given name."""  
    return f"Hello, {name}!"
```

4. Whitespace Rules

- **Around operators and commas:**

```
a = b + c
func(a, b, c)
```

- **Don't:**

```
a=b+c      # Bad
func( a , b , c )  # Bad
```

- **No extra spaces** inside parentheses or brackets.

```
✓ my_list = [1, 2, 3]
✗ my_list = [ 1, 2, 3 ]
```

5. Example of Good Style

```
def compute_average(scores):
    """Compute and return the average of a list of scores."""
    if not scores:
        return 0.0

    total = sum(scores)
    average = total / len(scores)
    return average

def main():
    """Prompt for scores and print the average."""
    scores = [float(x) for x in input("Enter scores separated by spaces: ").split()]
    avg = compute_average(scores)
    print(f"Average score: {avg:.2f}")

if __name__ == "__main__":
    main()
```